

A FAST HEURISTIC FOR TILE PARTITIONING AND PROCESSOR ASSIGNMENT IN HEVC

Panos K. Papadopoulos *

Maria Koziri †

Thanasis Loukopoulos *

*Dept. of Computer Science and Biomedical Informatics, University of Thessaly

†Dept. of Computer Science, University of Thessaly

ABSTRACT

As the compression efficiency of HEVC comes at the cost of high complexity, especially in the encoder's side, improved parallelization techniques that will speedup the encoding process are essential. One of the parallelization granules offered by HEVC is the tile level, whereby a frame is split into a grid like fashion with each resulting rectangular area (tile) being independently encoded. While tile parallelism has attracted research interest, the primary focus was to characterize performance and develop load balancing schemes assuming a one on one tile processor assignment. In this paper we target the problem of adaptively defining tile sizes (upon each frame) based on CTU cost estimation, under the assumption that the number of processors might be less than the number of tiles. It turns out that aside from the tile load balancing aspect, the problem has a processor scheduling sub-component that plays equal role. A fast algorithm is proposed that decides both tile sizing and tile processor assignment in an adaptive per frame fashion. Through experiments with common test sequences, the algorithm is shown to outperform the static tile sizing (one thread per tile) approach, by more than 30% (depending on the evaluation scenario) in terms of running time, without affecting video quality.

Index Terms— Tiles, partitioning, scheduling, video coding, HEVC

1. INTRODUCTION

In order to cope with the large computational demands of the 4K video coding process, HEVC (High Efficiency Video Coding) [1] introduced the possibility of splitting a frame into independently codeable rectangular areas called tiles. This split is done in a grid like fashion as illustrated by Fig. 1. Since tiles can be coded independently, they can be naturally processed in parallel by different processors/CPU cores.

Nevertheless, partitioning a frame into an excessive number of tiles might negatively affect video quality (not as much

though as with slices) [2]. Therefore, most existing works e.g. [3], [4], advocate a one on one tile-processor assignment in order to have the maximum speedup potential with the minimum negative effects. However, this approach is not always feasible and/or desirable. Consider for instance the case where (due to load from other tasks) the number of processors assigned to an encoding task is a prime number or, the case where fewer than the available cores are used for the encoding task in order to save power without crucially affecting time performance [5].

In this paper, we tackle the problem of resizing efficiently an $M \times N$ tile grid so that its assignment to $P \leq MN$ processors results in minimizing the load at the heaviest processor, thus, minimizing task execution time. Our primary contribution is a fast heuristic algorithm, called FAST (Fast Adaptive Scheduling of Tiles) that adaptively defines tile sizes upon every frame and assigns tiles to CPU cores for processing by creating one thread per core. FAST is shown to reduce running time by an average of roughly 20% (without affecting video quality) when compared with the default approach of using static uniform tile partitioning and spawning one thread per tile.

The rest of the paper is organized as follows. Section 2 provides a brief overview of the related work. Section 3 describes the FAST algorithm which is evaluated in Section 4. Finally, Section 5 includes our concluding remarks.

2. RELATED WORK

In [3] an overview of tiles in HEVC and the corresponding evaluation using static tile partitioning is presented. However, adjusting tile sizes in order to improve load balancing can lead to improved speedup. In [6] the boundaries of tiles are adjusted in per frame basis. At each frame, first the vertical tile grid lines and then the horizontal boundaries are determined by evenly balancing the expected workload at the vertical/horizontal zones induced. Workload estimates are based on average (from all previous frames) encoding time for each CTU. A similar approach but with static CTU weights was studied in [7], while in [4] an algorithm for setting tile boundaries was proposed, inspired by theoretical results on the array partitioning problem. The cost of each CTU in [4] was calculated by a sophisticated estimator that uses the GOP structure

Panos K. Papadopoulos was supported by scholarship from IKY (State Scholarships Foundation) funded by the Act "Strengthening Human Resources Research Potential via Doctorate Research" of the Operational Program "Human Resources Development Program, Education and Lifelong Learning", 2014-2020 co-financed by the European Social Fund (ESF) and the Greek Government.

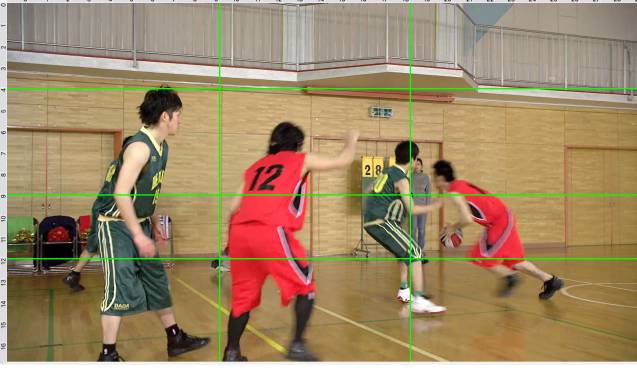


Fig. 1. BasketballDrive (frame 211, FAST partitioning)

of the encoded video sequence. We should mention that the same estimator is adopted in this paper. In the same context, Chan et al. in [8] propose two different CTU cost metrics, based on which tile boundaries are adjusted. The first one is the time of the corresponding CTU in the previous frame, whereas the other which is related to coding efficiency corresponds to the estimation of performance degradation when a tile boundary is located at the right-hand side of that CTU.

More closely related are the works where the number of available cores is not necessarily equal with the number of tiles. Shafique et. al in [5] propose a load balancing tile formation technique and a tile to core mapping. According to the characteristics of the video sequence to be encoded (i.e. frame resolution, frame rate) and of the system responsible for the encoding (i.e. processing power of the cores) the total number of tiles, along with the size of each tile, and the tile-processor assignment is determined. Nevertheless, these decisions are taken once at the beginning of each sequence, thus, missing the opportunities of further improvement through adaptive tile resizing and rescheduling upon every frame. Malossi et al. in [9] study the case where the number of available cores change during the encoding process. With the use of a static table (built off-line) associating different range of cores to specific tiling partitions, e.g., 2×2 , 3×3 etc., the number of tiles changes in a per-frame base according to the number of available cores. However, once the tile grid partitioning is defined, a uniform static approach for tile sizing is followed. This approach can be viewed as complimentary to ours in the sense that the FAST algorithm proposed in this paper can work with any tile grid dimensions, including the ones proposed in [9].

3. PROPOSED ALGORITHM

3.1. Preliminaries

In HEVC each frame is split into CTUs of maximum size 64×64 pixels, which are processed separately, in a raster-scan order. Thus, we can sufficiently characterize the total encoding time of a frame by estimating the time that will be

spent for compressing each CTU. As the problem of predicting CTU coding time is out of the scope of this paper, which focuses on the partitioning and scheduling aspects of tile parallelism, wherever a predictor is needed, LDE (LowDelay Estimator) proposed in [4], is used. LDE is an estimator that considers the hierarchical GOP structure used for the Low Delay (LD) common test condition [10] and was shown in [4] to achieve the best performance for such sequences.

For a frame F of size $height \times width$ (in CTUs) we can capture the encoding time required for it through a corresponding matrix A of dimensions $height \times width$, whereby its elements represent CTUs encoding times. By performing non-overlapping $M - 1$ horizontal cuts and $N - 1$ vertical ones the frame is effectively split into an $M \times N$ tile grid, with each tile having a corresponding cost equaling the aggregated weights of its elements i.e., the CTUs within the specific tile. Tiles should be scheduled for processing in P processors. The load of each processor is the aggregate cost of the tiles assigned to it. The makespan S of the schedule equals the cost of the most loaded processor. The problem tackled in the paper is as follows: Given A , P and $T = M \times N$ (and $T \geq P$), find $M - 1$ horizontal and $N - 1$ vertical cuts, and the tiles to processors assignment, such that S is minimized.

Algorithm 1 FAST Pseudocode

```

1:  $bestPartition \leftarrow Uniform$ 
2:  $bestSol \leftarrow MaxMin(tilePartition)$ 
3:  $found \leftarrow 1$ 
4: while  $found$  do
5:    $found \leftarrow 0$ 
6:    $candidateSol \leftarrow \infty$ 
7:    $candidatePartitions \leftarrow NULL$ 
8:    $candidateTiles \leftarrow heavyProcessorTiles$ 
9:   for  $x \in candidateTiles$  do
10:     $candidatePartitions \cup = \{x \text{ shrinkings}\}$ 
11:   end for
12:   for  $r \in candidatePartitions$  do
13:     $candidateSol \leftarrow MaxMin(r)$ 
14:    if  $candidateSol < bestSol$  then
15:       $update \ bestSol, bestPartition$ 
16:       $found \leftarrow 1$ 
17:    end if
18:   end for
19: end while
20:  $Implement(bestPartition)$ 

```

3.2. Algorithm Description

The FAST algorithm operates as follows. It starts with a uniform $M \times N$ tile grid and performs tile processor assignment using the $Max - Min$ approach [11], whereby the heaviest tile (Max) is assigned to the least loaded processor (Min) in an iterative fashion until all tiles are assigned. FAST attempts

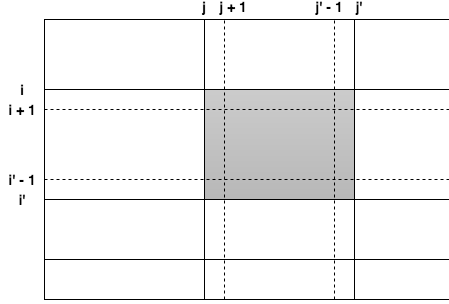


Fig. 2. Tile Resizing Options

to iteratively reduce the load of the heaviest processor by considering the shrinking of the tiles assigned to it. In particular for each tile assigned, four shrinking options are considered, one for each defining tile edge. Shrinking is attempted by reducing tile size by one CTU row or CTU column, depending on whether the shrinking is attempted on a horizontal or vertical edge respectively. Fig. 2 clarifies the aforementioned options by showing the four possible shrinking reductions with dashed lines.

Such tile boundary changes affect more tiles and not just the one upon which they were calculated. Therefore, upon each change considered, FAST recalculates tile costs and invokes MaxMin to obtain processor assignment. Among all candidate tile resizing and processor assignments (maximum $4 * \text{NumberOfTilesAtHeaviestProcessor}$) the algorithm selects the solution that improves current makespan the most. It then iterates the process, until no further improvement in makespan is possible by tile boundary reduction, at which point it implements the specific tile partitioning and tile-core assignment. The pseudocode of the algorithm is shown in Algorithm 1.

4. EXPERIMENTS

We implemented FAST tile parallelism using the HM 16.15 reference software [12] and OpenMP. We conducted experiments on a Linux Server with two 12-core Intel Xeon E5-2650 running at 2.20GHz. We used Class A and B test sequences as defined by common test conditions described in [10]. All the results were obtained assuming the LD scenario [10] with an initial I frame followed by P frames and a GOP size of 4. QP was set to 32, bit depth was 8, CTU size 64×64 , max depth for partitioning was set to 4 and search mode to TZ.

We conducted experiments for all test sequences with (a) 9 (3×3) tiles and 4-9 processors and (b) 12 (4×3) tiles and 4-12 processors. The performance of FAST was compared against a static uniform tile partitioning scheme that uses one thread per tile (Static). In all cases we measure the achievable speedup versus a sequential execution with one tile as follows: $\text{time}(\text{Sequential})/\text{time}(\text{Algorithm})$. Figs. 3,4

demonstrate the achievable speedup for 3×3 and 4×3 tile partitioning respectively.

As it can be observed FAST outperforms Static in all cases, with the performance difference being more apparent in the middle range (5-8 cores for the 3×3 grid and 5-11 cores for the 4×3 grid). In particular, the performance of Static suffers a large drop when the number of available cores is one less compared to the tile number, while FAST accounts for a much smoother performance degradation as the number of cores is reduced.

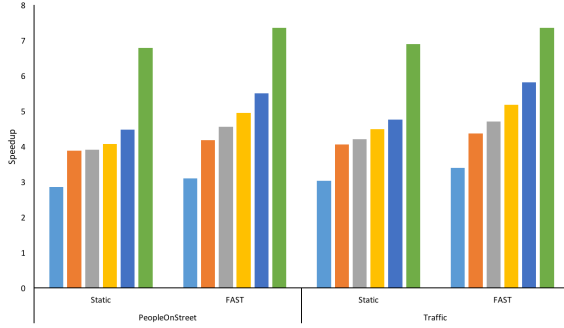
To better illustrate the performance of the FAST algorithm, Tables 1 and 2 summarize the percentage of reduction in execution time terms of FAST compared to Static, measured as: $(\text{time}(\text{Static}) - \text{time}(\text{FAST}))/\text{time}(\text{Static})$. As it is shown, in most cases FAST is roughly 20% faster than Static, while there are many cases where this percentage reaches more than 30%. It should also be noted that as shown by Table 2 the performance difference of the two algorithms tends to decrease when the number of cores is a divisor of the number of tiles (4, 6 and 12 cores). This is expected since the improvement margin from judicious processor scheduling narrows in these cases, e.g., with 4 cores, all cores will likely have an assignment of 3 tiles. Nevertheless, FAST manages to constantly outperform Static in these instances by a margin that reaches more than 10% in some cases. Furthermore, as shown in Table 3 (for the 3×3 scenario) FAST accounts for a negligible time overhead in the encoding process (the per frame encoding time was in the order of thousands or tens of thousands msecs depending on the number of cores and the sequence).

Finally, in order to assess the impact of the proposed algorithm in video compression efficiency, Tables 4 and 5 summarize the average (for all number of cores) PSNR (in dB) and average Bitrate (in bps) for each sequence and for 3×3 and 4×3 tile partitioning respectively. As it can be observed, with the exception of PeopleOnStreet in Table 4, where FAST even reduces bitrate by a rough 3%, all other differences are negligible.

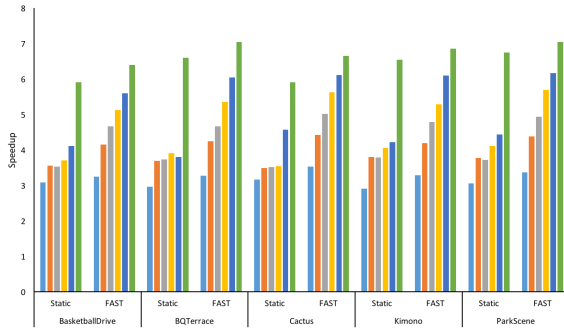
Summarizing the results, the achievable speedup of FAST, make it a clear winner against Static. It is also worth noting that for all the test cases with number of processors less than the total number of tiles the average time reduction of FAST over Static is roughly 20%, without any significant impact in video compression efficiency.

Table 1. TIME IMPROVEMENT (3X3 PARTITIONING)

Processors	Sequences	PeopleOnStreet	Traffic	BasketballDrive	BQTerrace	Cactus	Kimono	ParkScene
4		7.84%	10.83%	5.33%	9.48%	10.15%	11.41%	9.35%
5		7.36%	7.16%	14.21%	12.81%	20.90%	9.49%	14.02%
6		14.37%	10.63%	24.33%	20.19%	29.91%	20.92%	24.62%
7		17.83%	13.39%	27.66%	26.90%	37.16%	23.35%	27.76%
8		18.56%	18.22%	26.54%	37.17%	25.27%	30.93%	27.95%
9		7.66%	6.19%	7.61%	6.34%	11.22%	4.66%	4.23%



(a) Class A sequences



(b) Class B sequences

Fig. 3. Speedup for 3x3 tile partitioning

Table 2. TIME IMPROVEMENT (4X3 PARTITIONING)

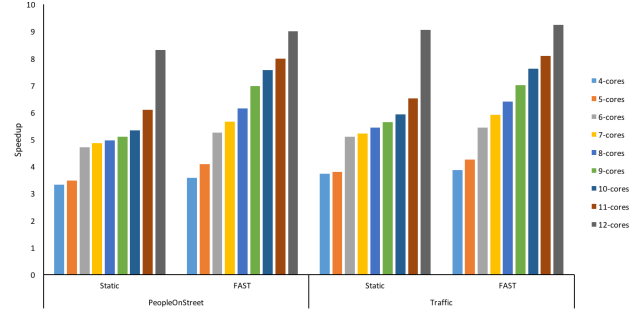
Processors	Sequences	PeopleOnStreet	Traffic	BasketballDrive	BQTerrace	Cactus	Kimono	ParkScene
4		7.02%	3.73%	4.71%	5.61%	8.55%	6.63%	7.14%
5		14.88%	10.54%	20.27%	19.25%	18.99%	19.44%	19.22%
6		10.07%	6.45%	9.71%	7.76%	12.88%	8.66%	10.15%
7		14.01%	11.49%	19.09%	12.72%	20.45%	12.93%	11.42%
8		19.10%	15.06%	21.14%	22.29%	21.20%	16.27%	19.60%
9		26.78%	19.48%	22.82%	28.61%	29.83%	28.91%	29.77%
10		29.48%	22.19%	25.19%	30.67%	34.04%	28.85%	32.72%
11		23.71%	19.49%	24.43%	37.73%	26.05%	31.58%	34.43%
12		7.70%	2.05%	5.70%	3.28%	2.85%	2.54%	1.91%

Table 3. TIME OVERHEAD (MSEC) OF FAST (3X3)

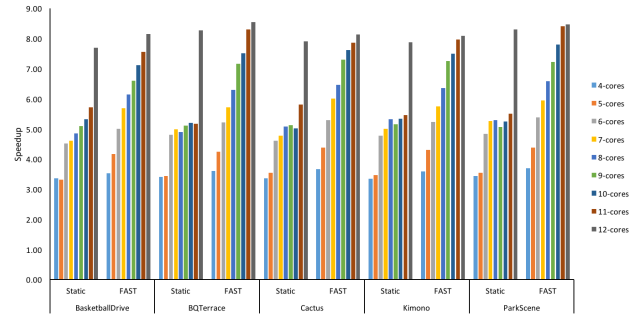
Sequences	Processors	4-cores	5-cores	6-cores	7-cores	8-cores	9-cores
PeopleOnStreet		0.082	0.068	0.076	0.083	0.081	0.031
Traffic		0.083	0.068	0.082	0.087	0.071	0.053
BasketballDrive		0.075	0.066	0.067	0.072	0.070	0.051
BQTerrace		0.077	0.066	0.077	0.084	0.090	0.046
Cactus		0.078	0.062	0.068	0.063	0.060	0.047
Kimono		0.081	0.064	0.078	0.078	0.089	0.046
ParkScene		0.080	0.061	0.074	0.081	0.075	0.045

5. CONCLUSIONS

In this paper we tackled the combined problem of both tile partitioning in an adaptive manner and scheduling the resulting tiles to the available processors that might be less than the number of tiles. The proposed algorithm and tile parallelism were implemented in HM software. Performance evaluation indicated a reduction in encoding time that reached 37% compared to static uniform tile partitioning.



(a) Class A sequences



(b) Class B sequences

Fig. 4. Speedup for 4x3 tile partitioning

Table 4. PSNR - BITRATE (3X3 PARTITIONING)

	PSNR (dB)		Bitrate (bps)	
	Static	FAST	Static	FAST
PeopleOnStreet	33.858	33.796	8221.395	7926.369
Traffic	35.498	35.489	1992.615	1991.762
BasketballDrive	34.892	34.879	990.120	990.137
BQTerrace	33.485	33.481	1397.234	1394.853
Cactus	34.182	34.184	1696.929	1697.059
Kimono	36.577	36.560	724.172	723.071
ParkScene	33.581	33.572	810.818	810.509
Average	34.582	34.566	2261.898	2219.109

Table 5. PSNR - BITRATE (4X3 PARTITIONING)

	PSNR (dB)		Bitrate (bps)	
	Static	FAST	Static	FAST
PeopleOnStreet	33.861	33.857	8231.861	8226.412
Traffic	35.491	35.492	1997.270	1997.356
BasketballDrive	34.888	34.879	997.101	995.694
BQTerrace	33.485	33.484	1404.578	1402.056
Cactus	34.182	34.181	1705.222	1703.531
Kimono	36.565	36.551	727.848	727.410
ParkScene	33.570	33.578	813.769	813.348
Average	34.577	34.575	2268.235	2266.544

6. REFERENCES

- [1] Gary J Sullivan, Jens Ohm, Woo-Jin Han, and Thomas Wiegand, "Overview of the high efficiency video coding (hevc) standard," *IEEE Transactions on circuits and systems for video technology*, vol. 22, no. 12, pp. 1649–

- 1668, 2012.
- [2] Chi Ching Chi, Mauricio Alvarez-Mesa, Ben Juurlink, Gordon Clare, Félix Henry, Stéphane Pateux, and Thomas Schierl, “Parallel scalability and efficiency of hevc parallelization approaches,” *IEEE Transactions on circuits and systems for video technology*, vol. 22, no. 12, pp. 1827–1838, 2012.
 - [3] Kiran Misra, Andrew Segall, Michael Horowitz, Shilin Xu, Arild Fuldseth, and Minhua Zhou, “An overview of tiles in hevc,” *IEEE Journal of selected topics in signal processing*, vol. 7, no. 6, pp. 969–977, 2013.
 - [4] Maria Koziri, Panos K Papadopoulos, Nikos Tziritas, Nikos Giachoudis, Thanasis Loukopoulos, Samee U Khan, and Georgios I Stamoulis, “Heuristics for tile parallelism in hevc,” in *Signal Processing Conference (EU-SIPCO), 2017 25th European*. IEEE, 2017, pp. 1514–1518.
 - [5] Muhammad Shafique, Muhammad Usman Karim Khan, and Jörg Henkel, “Power efficient and workload balanced tiling for parallelized high efficiency video coding,” in *Image Processing (ICIP), 2014 IEEE International Conference on*. IEEE, 2014, pp. 1253–1257.
 - [6] Iago Storch, Daniel Palomino, Bruno Zatt, and Luciano Agostini, “Speedup-aware history-based tiling algorithm for the hevc standard,” in *Image Processing (ICIP), 2016 IEEE International Conference on*. IEEE, 2016, pp. 824–828.
 - [7] Yong-Jo Ahn, Tae-Jin Hwang, Dong-Gyu Sim, and Woo-Jin Han, “Implementation of fast hevc encoder based on simd and data-level parallelism,” *EURASIP Journal on Image and Video Processing*, vol. 2014, no. 1, pp. 16, 2014.
 - [8] Wen-Jiin Tsai Chia-Hsin Chan, Chun-Chuan Tu, “Improve load balancing and coding efficiency of tiles in high efficiency video coding by adaptive tile boundary,” *Journal of Electronic Imaging*, vol. 26, pp. 26 – 26 – 10, 2017.
 - [9] Giovani Malossi, Daniel Palomino, Cláudio Diniz, Altamiro Susin, and Sergio Bampi, “Adjusting video tiling to available resources in a per-frame basis in high efficiency video coding,” in *New Circuits and Systems Conference (NEWCAS), 2016 14th IEEE International*. IEEE, 2016, pp. 1–4.
 - [10] Frank Bossen, “Common test conditions and software reference configurations,” in *Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG11, 5th meeting, Jan. 2011*, 2011.
 - [11] Tracy D Braun, Howard Jay Siegel, Noah Beck, Ladislau L Bölöni, Muthucumaru Maheswaran, Albert I Reuther, James P Robertson, Mitchell D Theys, Bin Yao, Debra Hensgen, et al., “A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems,” *Journal of Parallel and Distributed computing*, vol. 61, no. 6, pp. 810–837, 2001.
 - [12] “Hm 16.15 reference software, <http://hevc.hhi.fraunhofer.de>,” .